

Capsule-Based RRT-Connect for Dual-Arm Constrained Motion Planning

Algorithm Reference

Versioned Additions (Log-Linked)

This section is synchronized with `docs/DUAL_ARM_PLANNER_CHANGELOG.md`. Use the changelog as the index; use this document for mathematical/pseudocode details.

v0.1 – 2026-03-07

Tag: *Hierarchical Cartesian Dual-Arm Planner Baseline*

Stage hierarchy. Planner modes are explicitly separated by `(enable_ik, enable_collision)`:

Stage 1 : (0,0), Stage 1b : (0,1), Stage 2 : (1,0), Stage 3 : (1,1).

This isolates bottlenecks across task-space connectivity, IK feasibility, and collision feasibility.

Distance metric update. Default nearest-neighbor metric is now feature-point based:

$$\mathcal{D}_{feature}(T_1, T_2) = \|\Phi(T_2) - \Phi(T_1)\|_2,$$

where $\Phi(T)$ stacks world positions of selected bar feature points. Legacy `pose6d` remains available.

Guided sampling + warm start. Sampling now mixes goal/guide/uniform components with probabilities $\{p_g, p_u, 1 - p_g - p_u\}$. In Stage 3, a Stage-2 path can seed guide poses and warm-start smoothing/interpolation before full Stage-3 RRT.

Fast path and ladder fallback. After a task-space connection, planner first attempts projection chaining along the capsule pose chain:

$$\mathbf{q}_k = \Pi(\text{IK}_R(T_k T_R^{bar}, \mathbf{q}_{k-1,R}), \mathbf{q}_{k-1,L}).$$

If this fails, planner uses two-pass rung expansion and ladder shortest-path search.

Practical interpretation (why both exist):

- **Fast path** is an optimistic shortcut: once RRT connects in task space, try to recover the full 12-DOF joint trajectory directly by marching waypoint-to-waypoint with seed-chained IK and projection. If every step succeeds (and collisions pass in Stage 3), return immediately.

- **Ladder fallback** is the robust backup: if any fast-path step fails (IK, projection, or collision), expand each waypoint into multiple IK candidates (ladder rungs), build inter-rung edges by joint continuity, and solve for the best feasible chain.
- **Why this hybrid is faster:** easy/smooth cases avoid ladder-graph cost entirely; hard cases still remain solvable through combinatorial ladder search.

Performance changes.

- KD-tree nearest-neighbor for feature vectors in tree extension.
- IK expansion cache keyed by pose/expansion parameters.
- Collision cache keyed by quantized joint configuration.
- Capsule-path decimation before ladder expansion.
- Two-pass ladder expansion (small pass, then full pass if needed).
- Dynamic-programming shortest-path ladder solver (default).

Implementation pointers (current code):

- KD-tree nearest-neighbor: `trajectory_dual_cart_constrained_solver.py:183--196`
- IK expansion cache: `trajectory_dual_cart_constrained_solver.py:1658--1670`
- Collision cache: `trajectory_dual_cart_constrained_solver.py:1416--1422`
- Capsule-path decimation: `trajectory_dual_cart_constrained_solver.py:1446--1457`
- Two-pass ladder expansion: `trajectory_dual_cart_constrained_solver.py:1707--1743`
- DP shortest-path ladder solver: `trajectory_dual_cart_constrained_solver.py:353--444`

Profiling and failure attribution. Internal profiler now reports operation-level timing. Failure counters distinguish:

- RRT task-space failure (no tree connection),
- ladder failure (connected in task-space but no feasible joint chain).

Overview

Rather than planning directly in the full 12-DOF joint space, this planner operates in **6D task space** (the SE(3) pose of a grasped bar object), while switching behavior by stage:

Stage 1 : (`-enable_ik`, `-enable_collision`), Stage 1b : (`-enable_ik`, `enable_collision`), Stage 2 : (`enable_ik`,

A *Capsule* node stores a bar pose and an optional list of dual-arm IK configurations. RRT-Connect grows two trees of Capsules in task space; after connection, the planner first attempts projection-chain recovery (fast path), then falls back to ladder expansion + graph search only when needed.

Key notation:

- $\mathbf{q} \in \mathbb{R}^{12}$: concatenated joint configuration $[\mathbf{q}_L \mid \mathbf{q}_R]$.

- T_A^B : rigid transform of frame A expressed in frame B .
- $\mathcal{C} = (\mathbf{p}, Q, \mathcal{Q}, \pi)$: a Capsule with bar position $\mathbf{p}$, quaternion Q , config list $\mathcal{Q}$, and parent pointer π .
- $\text{FK}(\cdot), \text{IK}(\cdot)$: forward/inverse kinematics.
- $d(\mathbf{q}_1, \mathbf{q}_2) = \|\Delta\theta(\mathbf{q}_1, \mathbf{q}_2)\|_2$: joint-space distance with angle wrapping.

Highlight convention: blue lines denote additions/changes in the current implementation.

Algorithm 1 – Top-Level Planner

Purpose: orchestrate staged planning, warm-start shortcuts, and final post-processing.

Algorithm 1 PLAN($\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{goal}}, \text{stage}$)**Input:** Start/goal configs, stage flags, runtime knobs (`max_time`, `max_iterations`, `max_attempts`)**Output:** Joint-space path Π or FAILURE

— *Step 1: Initialize global bar $\leftarrow$ right transform* —

- 1: Set robot to $\mathbf{q}_{\text{goal}}$
- 2: $T_{\text{bar}}^W \leftarrow \text{FK}_{\text{bar}}(\mathbf{q}_{\text{goal}})$
- 3: $T_R^{\text{bar}} \leftarrow (T_{\text{bar}}^W)^{-1} \cdot T_R^W$ $\triangleright$ fixed offset: bar frame $\rightarrow$ right tool

— *Step 2: Compute start and goal bar poses* —

- 4: $T_{\text{bar},s}^W \leftarrow T_{R,s}^W \cdot (T_R^{\text{bar}})^{-1}$
- 5: $T_{\text{bar},g}^W \leftarrow T_{R,g}^W \cdot (T_R^{\text{bar}})^{-1}$
- 6: (`enable_ik`, `enable_collision`) $\leftarrow$ DECODESTAGE(`stage`)
- 7: **if** `enable_ik` **then**
- 8: $\mathcal{Q}_s \leftarrow \text{CREATEVALIDCONFS}(T_{\text{bar},s}^W, T_R^{\text{bar}}, \delta = \text{start_goal_delta})$
- 9: $\mathcal{Q}_g \leftarrow \text{CREATEVALIDCONFS}(T_{\text{bar},g}^W, T_R^{\text{bar}}, \delta = \text{start_goal_delta})$
- 10: **else**
- 11: $\mathcal{Q}_s \leftarrow [\mathbf{q}_{\text{start}}], \mathcal{Q}_g \leftarrow [\mathbf{q}_{\text{goal}}]$
- 12: **end if**

— *Step 4: Build start and goal Capsules* —

- 13: $\mathcal{C}_s \leftarrow \text{CAPSULE}(T_{\text{bar},s}^W, \mathcal{Q}_s)$
- 14: $\mathcal{C}_g \leftarrow \text{CAPSULE}(T_{\text{bar},g}^W, \mathcal{Q}_g)$
- 15: ATTACHFEATUREVEC($\mathcal{C}_s$); ATTACHFEATUREVEC($\mathcal{C}_g$)
- 16: $\Pi_{\text{warm}} \leftarrow \emptyset, \mathcal{G}_{\text{guide}} \leftarrow \emptyset$
- 17: **if** `stage` = 3 **then**
- 18: $\Pi_{\text{warm}} \leftarrow \text{PLANSTAGE2SEED}(\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{goal}})$
- 19: $\mathcal{G}_{\text{guide}} \leftarrow \text{GUIDEPOSESFROMPATH}(\Pi_{\text{warm}})$
- 20: **end if**
- 21: **for** $a = 1$ **to** `max_attempts` **do**
- 22: **if** `enable_collision` $\wedge (\Pi_{\text{warm}} \neq \emptyset) \wedge \text{warm_start_first}$ **then**
- 23: $\Pi \leftarrow \text{SMOOTHANDINTERPOLATE}(\Pi_{\text{warm}})$
- 24: **if** $\Pi \neq \emptyset$ **then return** Π
- 25: **end if**
- 26: **end if**
- 27: $\Pi \leftarrow \text{RRTCONNECTCAPSULE}(\mathcal{C}_s, \mathcal{C}_g, \text{enable_ik}, \text{enable_collision}, \mathcal{G}_{\text{guide}})$
- 28: **if** $\Pi = \emptyset$ **then**
- 29: **continue**
- 30: **end if**
- 31: **if** $\neg \text{enable_ik}$ **then**
- 32: **if return_task_path** **then return** raw task-space poses
- 33: **continue**
- 34: **end if**
- 35: $\Pi \leftarrow \text{JOINTSPACE SMOOTH}(\Pi, \text{CSPACEEXTENDWITHPROJECTION})$
- 36: $\Pi \leftarrow \text{INTERPOLATE}(\Pi, \text{CSPACEEXTENDWITHPROJECTION})$
- 37: **if** $\Pi \neq \emptyset$ **then return** Π
- 38: **end if**
- 39: **end for**
- 40: **return** FAILURE

Algorithm 2 – RRT-Connect in Capsule Space

Purpose: connect start/goal in task space and recover a feasible joint path via fast path or ladder fallback. What “merge” means here: a successful BiRRT connection only means that the forward and backward trees have met in *bar-pose space*. It does *not* mean that the IK branch carried by the forward tree is already compatible with the IK branch carried by the backward tree. The planner therefore treats the merged capsule chain as a task-space candidate only, then tries to recover a single consistent joint-space chain with TRYPROJECTCHAIN. If that fails, it expands each rung to multiple IK candidates with EXPANDCARTPATHLADDERGRAPH and resolves branch incompatibility through LADDERGRAPHSEARCH.

Algorithm 2 RRTCONNECTCAPSULE($\mathcal{C}_s, \mathcal{C}_g$)

Input: Start/goal capsules, stage flags, draw/profiler handles

Output: Ordered joint-config sequence or $\emptyset$

```
1:  $\mathcal{T}_1 \leftarrow \{\mathcal{C}_s\}; \mathcal{T}_2 \leftarrow \{\mathcal{C}_g\}$ 
2: for  $k = 1$  to  $N_{\max}$  do
    — Swap: grow from the smaller tree —
3:   if  $|\mathcal{T}_1| > |\mathcal{T}_2|$  then swap( $\mathcal{T}_1, \mathcal{T}_2$ )
4:   end if
    — Sample (goal/guide/uniform mixture) —
5:    $\mathcal{C}_{\text{rand}} \leftarrow \text{SAMPLEMIXED}()$ 
    — Extend  $\mathcal{T}_1$  toward  $\mathcal{C}_{\text{rand}}$  —
6:    $\mathcal{C}_{\text{new}}, \mathcal{T}_1 \leftarrow \text{CARTEXTENDTOWARD}(\mathcal{T}_1, \mathcal{C}_{\text{rand}}, \text{enable\_ik})$ 
    — Attempt to connect  $\mathcal{T}_2$  to  $\mathcal{C}_{\text{new}}$  —
7:    $\mathcal{C}_{\text{last}}, \text{success}, \mathcal{T}_2 \leftarrow \text{CARTEXTENDTOWARD}(\mathcal{T}_2, \mathcal{C}_{\text{new}}, \text{enable\_ik})$ 
8:   if success then
    — Reconstruct raw capsule path —
9:      $\Pi_1 \leftarrow \text{RETRACE}(\mathcal{C}_{\text{new}}); \Pi_2 \leftarrow \text{RETRACE}(\mathcal{C}_{\text{last}})$ 
10:     $\Pi_c \leftarrow \Pi_1 \parallel \text{reverse}(\Pi_2)$ 
11:     $\Pi_c \leftarrow \text{DEDUPLICATE}(\Pi_c)$  ▷ remove duplicate bar poses
12:    if  $\neg \text{enable\_ik}$  then
13:      if return\_task\_path then return  $\text{CHECKCAPSULEPATH}(\Pi_c)$ 
14:      continue
15:    end if
16:     $\Pi_c \leftarrow \text{DECIMATECAPSULEPATH}(\Pi_c)$ 
17:     $\Pi_{\text{fast}} \leftarrow \text{TRYPROJECTCHAIN}(\Pi_c, \text{enable\_collision})$ 
18:    if  $\Pi_{\text{fast}} \neq \emptyset$  then return  $\Pi_{\text{fast}}$ 
19:    end if
20:    if all rungs in  $\Pi_c$  already expanded then
21:      continue
22:    end if
23:     $\hat{\Pi}_c \leftarrow \text{EXPANDCARTPATHLADDERGRAPH}(\Pi_c)$ 
24:     $\text{results} \leftarrow \text{LADDERGRAPHSEARCH}(\hat{\Pi}_c, \text{mode})$ 
25:    if  $\text{results} = \emptyset$  then
26:      continue
27:    end if
28:    return best result from ladder search
29:  end if
30: end for
31: return  $\emptyset$ 
```

Algorithm 2a – Deduplicate Capsule Path

Purpose: remove repeated bar poses while preserving a valid parent chain.

Algorithm 3 DEDUPLICATE(Π_c)

Input: Raw capsule sequence $\Pi_c = [\mathcal{C}_0, \dots, \mathcal{C}_m]$ **Output:** Pose-unique capsule sequence Π_u

```
1:  $\Pi_u \leftarrow []$ ;  $\mathcal{S}_{seen} \leftarrow \emptyset$ ;  $\pi \leftarrow \emptyset$ 
2: for each  $\mathcal{C} \in \Pi_c$  do
3:    $k \leftarrow \text{POSEKEY}(\mathcal{C}.\text{pose}, \text{round\_decimals})$ 
4:   if  $k \in \mathcal{S}_{seen}$  then
5:     continue
6:   end if
7:    $\mathcal{S}_{seen} \leftarrow \mathcal{S}_{seen} \cup \{k\}$ 
8:    $\hat{\mathcal{C}} \leftarrow \text{DEEPCOPY}(\mathcal{C})$ 
9:    $\hat{\mathcal{C}}.\text{parent} \leftarrow \pi$ 
10:   $\hat{\mathcal{C}}.\text{origin} \leftarrow \mathcal{C}$ 
11:  Append  $\hat{\mathcal{C}}$  to  $\Pi_u$ 
12:   $\pi \leftarrow \hat{\mathcal{C}}$ 
13: end for
14: return  $\Pi_u$ 
```

Algorithm 2b – Decimate Capsule Path*Purpose: reduce redundant waypoints before expensive ladder expansion.*

Algorithm 4 DECIMATECAPSULEPATH($\Pi_u, \tau_p, \tau_\theta$)

Input: Deduplicated sequence Π_u , position threshold τ_p , orientation threshold τ_θ **Output:** Reduced sequence Π_d

```
1: if  $|\Pi_u| \leq 2$  then
2:   return  $\Pi_u$ 
3: end if
4:  $\Pi_d \leftarrow [\Pi_u[0]]$ 
5: for  $i = 1$  to  $|\Pi_u| - 2$  do
6:    $\mathcal{C}_{prev} \leftarrow \Pi_d[-1]$ ;  $\mathcal{C}_{cur} \leftarrow \Pi_u[i]$ ;  $\mathcal{C}_{next} \leftarrow \Pi_u[i + 1]$ 
7:    $\Delta p \leftarrow \|\mathbf{p}_{next} - \mathbf{p}_{prev}\|_2$ 
8:    $\Delta \theta \leftarrow \angle(Q_{next}, Q_{prev})$ 
9:   if  $\Delta p < \tau_p$  and  $\Delta \theta < \tau_\theta$  then
10:    skip  $\mathcal{C}_{cur}$ 
11:   else
12:    Append  $\mathcal{C}_{cur}$  to  $\Pi_d$ 
13:   end if
14: end for
15: Append  $\Pi_u[-1]$  to  $\Pi_d$ 
16: return  $\Pi_d$ 
```

Algorithm 2c – Fast Projection Chain Recovery*Purpose: attempt direct seed-chained IK+projection recovery without building a full ladder graph.*

Algorithm 5 TRYPROJECTCHAIN(Π_d , enable_collision)

Input: Decimated capsule path Π_d , collision flag

Output: Joint path Π_q or $\emptyset$

```
1: if  $\Pi_d = \emptyset$  or  $\Pi_d[0].\mathcal{Q} = \emptyset$  then
2:   return  $\emptyset$ 
3: end if
4:  $\mathbf{q}_{prev} \leftarrow \Pi_d[0].\mathcal{Q}[0]; \quad \Pi_q \leftarrow [\mathbf{q}_{prev}]$ 
5:  $\mathbf{q}_{R,seed} \leftarrow \mathbf{q}_{prev}[6:]; \quad \mathbf{q}_{L,seed} \leftarrow \mathbf{q}_{prev}[:6]$ 
6: for each  $\mathcal{C} \in \Pi_d[1:]$  do
7:    $T_R^W \leftarrow \mathcal{C}.pose \cdot T_R^{\text{bar}}$ 
8:    $\mathbf{q}_R \leftarrow \text{IK}_R(T_R^W, \mathbf{q}_{R,seed})$ 
9:   if  $\mathbf{q}_R = \emptyset$  then
10:    return  $\emptyset$ 
11:   end if
12:    $\mathbf{q} \leftarrow \text{PROJECT}(\mathbf{q}_R, \mathbf{q}_{L,seed})$ 
13:   if  $\mathbf{q} = \emptyset$  then
14:    return  $\emptyset$ 
15:   end if
16:   if enable_collision and INCOLLISION( $\mathbf{q}$ ) then
17:    return  $\emptyset$ 
18:   end if
19:   Append  $\mathbf{q}$  to  $\Pi_q$ 
20:    $\mathbf{q}_{R,seed} \leftarrow \mathbf{q}[6:]; \quad \mathbf{q}_{L,seed} \leftarrow \mathbf{q}[:6]$ 
21: end for
22: return  $\Pi_q$ 
```

Algorithm 3 – Extend Toward a Target Capsule

Purpose: grow one tree branch toward a target capsule with stage-aware IK/collision behavior.

Algorithm 6 CARTEXTENDTOWARD($\mathcal{T}$, $\mathcal{C}_{\text{target}}$, enable_ik)

Input: Tree $\mathcal{T}$, target capsule $\mathcal{C}_{\text{target}}$ **Output:** Last added capsule $\mathcal{C}_{\text{last}}$, success flag, updated $\mathcal{T}$

```
1:  $\mathcal{C}_{\text{near}} \leftarrow \text{NEARESTNODE}(\mathcal{T}, \mathcal{C}_{\text{target}})$ 
2:   (KD-tree on feature vectors when available; fallback to linear scan)
3: Waypoints  $\leftarrow \text{CARTLINEARINTERP}(\mathcal{C}_{\text{near}}, \mathcal{C}_{\text{target}})$   $\triangleright$  SLERP + linear position, Alg. 5
4:  $\mathcal{C}_{\text{prev}} \leftarrow \mathcal{C}_{\text{near}}$ ; all_safe  $\leftarrow \text{True}$ 
5: for each pose  $T_{\text{bar}}^W$  in Waypoints do
6:   if enable_ik then
7:      $T_R^W \leftarrow T_{\text{bar}}^W \cdot T_R^{\text{bar}}$ 
8:      $\mathbf{q}_R \leftarrow \text{IK}_R(T_R^W, \text{seed} = \mathcal{C}_{\text{prev}}.\mathcal{Q}[0][6 :])$ 
9:     if  $\mathbf{q}_R \neq \emptyset$  then
10:       $\mathbf{q} \leftarrow \text{PROJECT}(\mathbf{q}_R, \mathcal{C}_{\text{prev}}.\mathcal{Q}[0][: 6])$ 
11:     else
12:       $\mathbf{q} \leftarrow \emptyset$ 
13:     end if
14:     if  $\mathbf{q} = \emptyset$  or INCOLLISION( $\mathbf{q}$ ) then
15:       all_safe  $\leftarrow \text{False}$ ;
16:       break
17:     end if
18:      $\mathcal{C}_{\text{new}} \leftarrow \text{CAPSULE}(T_{\text{bar}}^W, [\mathbf{q}], \text{parent} = \mathcal{C}_{\text{prev}})$ 
19:   else
20:      $\mathcal{C}_{\text{new}} \leftarrow \text{CAPSULE}(T_{\text{bar}}^W, \text{config} = \emptyset, \text{parent} = \mathcal{C}_{\text{prev}})$ 
21:   end if
22:    $\mathcal{T} \leftarrow \mathcal{T} \cup \{\mathcal{C}_{\text{new}}\}$ ;  $\mathcal{C}_{\text{prev}} \leftarrow \mathcal{C}_{\text{new}}$ 
23: end for
24: return  $\mathcal{C}_{\text{prev}}$ , all_safe,  $\mathcal{T}$ 
```

Algorithm 3a – Nearest Node Selection (NearestNode)*Purpose: choose nearest tree node using KD-tree acceleration when feature vectors are available.*

Algorithm 7 NEARESTNODE($\mathcal{T}$, $\mathcal{C}_{\text{target}}$)

Input: Tree $\mathcal{T}$ of capsules, target capsule $\mathcal{C}_{\text{target}}$ **Output:** Nearest capsule $\mathcal{C}_{\text{near}} \in \mathcal{T}$

```
1:  $\mathbf{v}_t \leftarrow \text{FEATUREVEC}(\mathcal{C}_{\text{target}})$ 
2: if  $\mathbf{v}_t \neq \emptyset$  and all nodes in  $\mathcal{T}$  have valid feature vectors then
3:   Build matrix  $V \leftarrow [\mathbf{v}_1, \dots, \mathbf{v}_{|\mathcal{T}|}]^\top$ 
4:    $\mathcal{K} \leftarrow \text{KD TREE}(V)$ 
5:    $i^* \leftarrow \mathcal{K}.\text{QUERYNEAREST}(\mathbf{v}_t)$ 
6:   return  $\mathcal{T}[i^*]$ 
7: else
8:    $\mathcal{C}_{\text{near}} \leftarrow \arg \min_{\mathcal{C} \in \mathcal{T}} \|\text{CARTDIST}(\mathcal{C}, \mathcal{C}_{\text{target}})\|_2$ 
9:   return  $\mathcal{C}_{\text{near}}$ 
10: end if
```

Algorithm 4 – Dual-Arm Constraint Projection

Purpose: recover full dual-arm configuration from right-arm state under rigid relative-pose constraint.

Algorithm 8 PROJECT($\mathbf{q}_R, \mathbf{q}_{L,\text{seed}}$)

Input: Right-arm config $\mathbf{q}_R \in \mathbb{R}^6$, left-arm seed $\mathbf{q}_{L,\text{seed}} \in \mathbb{R}^6$

Output: Full 12-DOF config $\mathbf{q}$ or $\emptyset$

```

1:  $T_R^W \leftarrow \text{FK}_R(\mathbf{q}_R)$ 
2:  $T_L^W \leftarrow T_R^W \cdot (T_L^R)$   $\triangleright T_L^R$  is the precomputed rigid offset (constant)
3:  $\mathbf{q}_L \leftarrow \text{IK}_L(T_L^W, \text{seed} = \mathbf{q}_{L,\text{seed}})$ 
4: if  $\mathbf{q}_L = \emptyset$  then
5:   return  $\emptyset$ 
6: end if
7: return  $[\mathbf{q}_L \mid \mathbf{q}_R]$ 

```

Algorithm 5 – Cartesian Linear Interpolation (SLERP)

Purpose: generate pose waypoints with linear position interpolation and shortest-arc quaternion SLERP.

Algorithm 9 CARTLINEARINTERP($\mathcal{C}_1, \mathcal{C}_2$)

Input: Capsules $\mathcal{C}_1, \mathcal{C}_2$ with poses $(\mathbf{p}_1, Q_1)$ and $(\mathbf{p}_2, Q_2)$

Output: Sequence of waypoint poses

```

1:  $\Delta p \leftarrow \|\mathbf{p}_2 - \mathbf{p}_1\|$ ;  $\Delta \theta \leftarrow \angle(Q_1, Q_2)$ 
2:  $N \leftarrow \lceil \max(\Delta p / r_p, \Delta \theta / r_\theta) \rceil$   $\triangleright r_p=5\text{cm}, r_\theta=0.1\text{ rad}$ 
3: if  $Q_1 \cdot Q_2 < 0$  then  $Q_2 \leftarrow -Q_2$ 
4: end if  $\triangleright$  ensure shortest-arc SLERP
5: Waypoints  $\leftarrow []$ 
6: for  $i = 0$  to  $N$  do
7:    $t \leftarrow i/N$ 
8:    $\mathbf{p}(t) \leftarrow (1-t)\mathbf{p}_1 + t\mathbf{p}_2$ 
9:    $Q(t) \leftarrow \text{Slerp}(Q_1, Q_2, t)$ 
10:  Waypoints  $\leftarrow \text{Waypoints} \cup \{(\mathbf{p}(t), Q(t))\}$ 
11: end for
12: return Waypoints

```

Algorithm 6 – Ladder Graph Construction (ExpandCartPathLadderGraph)

Purpose: expand capsule rungs to multi-IK sets and build inter-rung connectivity for graph search.

Algorithm 10 EXPANDCARTPATHLADDERGRAPH(Π_c)

Input: Deduplicated capsule sequence $\Pi_c = [\mathcal{C}_0, \dots, \mathcal{C}_n]$

Output: Expanded ladder graph $\hat{\Pi}_c$ with inter-rung connectivity

```
1: — Pass 1 (small): cheap expansion where missing —
2: for each capsule  $\mathcal{C}_i$  in  $\Pi_c$  do
3:   if  $|\mathcal{C}_i.Q| \leq 1$  then
4:      $\mathcal{C}_i.Q \leftarrow \text{EXPANDSINGLE}(\mathcal{C}_i, \delta = \min(\text{ladder\_expand\_delta}, \pi/6), \text{attempts} = \text{small})$ 
5:   end if
6:   Append  $\mathcal{C}_i$  to  $\hat{\Pi}_c$ 
7: end for
8: Build connections across adjacent rungs
9: if at least one inter-rung edge exists then return  $\hat{\Pi}_c$ 
10: end if
11: — Pass 2 (full): retry only unresolved rungs —
12: for each capsule  $\mathcal{C}_i$  in  $\hat{\Pi}_c$  do
13:   if  $|\mathcal{C}_i.Q| \leq 1$  then
14:      $\mathcal{C}_i.Q \leftarrow \text{EXPANDSINGLE}(\mathcal{C}_i, \delta = \text{ladder\_expand\_delta}, \text{attempts} = \text{full})$ 
15:   end if
16: end for
17: Rebuild inter-rung connection map
18: return  $\hat{\Pi}_c$ 
```

Algorithm 6a – Single-Rung Expansion (ExpandSingle)

Purpose: expand one rung with cache-aware multi-IK generation.

Algorithm 11 EXPANDSINGLE($\mathcal{C}, \delta, \text{max_attempts}$)

Input: Capsule $\mathcal{C}$ with pose T_{bar}^W , expansion sweep δ , attempt budget

Output: Expanded capsule $\hat{\mathcal{C}}$ with multi-IK rung configs

```
1:  $k \leftarrow (\text{POSEKEY}(T_{\text{bar}}^W), \delta, \text{max\_attempts})$ 
2: if  $k \in \text{ik\_cache}$  then
3:    $\mathcal{Q} \leftarrow \text{ik\_cache}[k]$ 
4: else
5:    $\mathcal{Q} \leftarrow \text{CREATEVALIDCONFS}(\text{IK}_R, T_{\text{bar}}^W, T_R^{\text{bar}}, \delta, \text{max\_attempts}, \text{collision\_fn})$ 
6:    $\text{ik\_cache}[k] \leftarrow \mathcal{Q}$ 
7: end if
8: return CAPSULE( $T_{\text{bar}}^W, \mathcal{Q}, \text{parent} = \emptyset$ )
```

Algorithm 6b – Multi-IK Generation (CreateValidConfs)

Purpose: generate diverse feasible dual-arm configurations around a bar pose via yaw perturbation + projection.

Algorithm 12 CREATEVALIDCONFS($\text{IK}_R, T_{\text{bar}}^W, T_R^{\text{bar}}, \delta, \text{max_attempts}$)

Input: Right-arm IK handle, bar pose, bar→right transform, yaw sweep δ , attempt count

Output: Unique feasible 12-DOF configuration set $\mathcal{Q}$ or $\emptyset$

```
1:  $\mathcal{Q}_{\text{raw}} \leftarrow []$ 
2:  $\Theta \leftarrow \text{Linspace}(0, \delta, \text{max\_attempts})$ 
3: for each  $\theta \in \Theta$  do
4:    $\mathbf{q}_R^{\text{seed}} \leftarrow \text{UNIFORM}([- \pi, \pi]^6)$ 
5:    $\Delta T \leftarrow \text{POSE}(\text{yaw} = \theta)$ 
6:    $\tilde{T}_{\text{bar}}^W \leftarrow T_{\text{bar}}^W \cdot \Delta T$ 
7:    $T_R^W \leftarrow \tilde{T}_{\text{bar}}^W \cdot T_R^{\text{bar}}$ 
8:    $\mathbf{q}_R \leftarrow \text{IK}_R(T_R^W, \mathbf{q}_R^{\text{seed}})$ 
9:   if  $\mathbf{q}_R \neq \emptyset$  then
10:     $\mathbf{q}_R \leftarrow \text{NORMALIZEToPi}(\mathbf{q}_R)$ 
11:     $\mathcal{Q}_{\text{proj}} \leftarrow \text{PROJECTMULTIPLE}(\mathbf{q}_R, \text{max\_attempts}, \text{collision\_fn})$ 
12:    if  $\mathcal{Q}_{\text{proj}} \neq \emptyset$  then
13:      Append all elements of  $\mathcal{Q}_{\text{proj}}$  to  $\mathcal{Q}_{\text{raw}}$ 
14:      if immediate is True then
15:        return first element in  $\mathcal{Q}_{\text{proj}}$ 
16:      end if
17:    end if
18:  end if
19: end for
20:  $\mathcal{Q} \leftarrow \text{DEDUPLICATEBYALLCLOSE}(\mathcal{Q}_{\text{raw}}, \text{atol} = 10^{-2})$ 
21: if  $|\mathcal{Q}| = 0$  then
22:   return  $\emptyset$ 
23: end if
24: return  $\mathcal{Q}$ 
```

Algorithm 7 – Ladder Graph Search (LadderGraphSearch)

Purpose: solve the expanded ladder either by shortest-path mode or full DFS enumeration mode.

Algorithm 13 LADDERGRAPHSEARCH($\hat{\Pi}_c, \text{mode}$)

Input: Expanded ladder $\hat{\Pi}_c = [\mathcal{C}_0, \dots, \mathcal{C}_n]$ with connection maps

Output: Feasible joint-config path candidates

```
1: Build adjacency  $E_\ell$ : for each rung-pair  $(\ell, \ell+1)$ ,  $E_\ell[j] \leftarrow \{k \mid k \in \mathcal{C}_{\ell+1}.\text{connection}, j \in \mathcal{C}_{\ell+1}.\text{connection}[k]\}$ 
2: if mode = shortest then
3:   return  $\text{DPShortestPath}(\hat{\Pi}_c)$ 
4: else
5:   return  $\text{EnumerateAllPathsDFS}(\hat{\Pi}_c)$ 
6: end if
```

Algorithm 8 – Warm-Start Shortcut (SmoothAndInterpolate)

Purpose: cheaply validate and refine a Stage-2 seed path before full Stage-3 search.

Algorithm 14 SMOOTHANDINTERPOLATE(Π_{warm})

Input: Warm-start joint path Π_{warm}

Output: Collision-feasible refined path or $\emptyset$

```
1: if  $\Pi_{\text{warm}} = \emptyset$  or  $|\Pi_{\text{warm}}| < 2$  then
2:   return  $\emptyset$ 
3: end if
4:  $\Pi_s \leftarrow \text{SMOOTHWITHCOLLISION}(\Pi_{\text{warm}}, \text{CSpaceExtendWithProjection}, \text{max\_iters})$ 
5:  $\Pi_r \leftarrow \text{INTERPOLATE}(\Pi_s, \text{CSpaceExtendWithProjection})$ 
6: if  $\Pi_r \neq \emptyset$  and  $|\Pi_r| > 1$  then
7:   return  $\Pi_r$ 
8: end if
9: return  $\emptyset$ 
```

Algorithm 9 – Joint-Space Smoothing

Purpose: shorten a feasible path with collision-safe random shortcutting.

Algorithm 15 JOINTSPACESMOOTH(Π , EXTENDFN, N_{iter})

Input: Joint-config path Π , extension function, iteration count N_{iter}

Output: Shorter path (collision-feasible shortcutting)

```
1: for  $i = 1$  to  $N_{\text{iter}}$  do
2:    $(i, j) \leftarrow$  two random indices from  $\{0, \dots, |\Pi| - 1\}$  with  $j - i > 1$ 
3:   segment  $\leftarrow \text{CSpaceExtendWithProjection}(\Pi[i], \Pi[j])$ 
4:   if segment is valid and all intermediate states pass collision then
5:      $\Pi \leftarrow \Pi[:i] \parallel \text{segment} \parallel \Pi[j:]$ 
6:   end if
7: end for
8: return  $\Pi$ 
```

Algorithm 10 – C-Space Extension via Projection

Purpose: interpolate right-arm motion while re-projecting left-arm states to preserve the dual-arm constraint.

Algorithm 16 CSPACEEXTENDWITHPROJECTION($\mathbf{q}_1, \mathbf{q}_2$)

Input: Start config $\mathbf{q}_1$, end config $\mathbf{q}_2$

Output: Sequence of 12-DOF configs interpolating right arm, re-projecting left arm

```
1:  $\Delta\theta_R \leftarrow \text{AngleDist}(\mathbf{q}_1[6:], \mathbf{q}_2[6:])$ 
2:  $N \leftarrow \max(1, \lceil \|\Delta\theta_R/r_{\text{joint}}\|_2 \rceil)$   $\triangleright r_{\text{joint}} = 5^\circ$  per joint
3:  $\mathbf{q}_{\text{prev}} \leftarrow \mathbf{q}_1$ 
4: for  $i = 0$  to  $N$  do
5:    $t \leftarrow i/N$ 
6:    $\mathbf{q}_R^{(i)} \leftarrow \text{normalize}(\mathbf{q}_1[6:] + t \Delta\theta_R)$ 
7:    $\mathbf{q}^{(i)} \leftarrow \text{PROJECT}(\mathbf{q}_R^{(i)}, \mathbf{q}_{\text{prev}}[:6])$ 
8:   if  $\mathbf{q}^{(i)} \neq \emptyset$  then
9:     yield  $\mathbf{q}^{(i)}$ ;  $\mathbf{q}_{\text{prev}} \leftarrow \mathbf{q}^{(i)}$ 
10:  else
11:    yield  $\emptyset$ ;
12:    break
13:  end if
14: end for
```

Algorithm 11 – Task-Space Distance

Purpose: compute nearest-neighbor distance features in either feature or pose6d metric mode.

Algorithm 17 CARTDIST($\mathcal{C}_1, \mathcal{C}_2, \text{metric}$)

Input: Two Capsules with poses $(R_1, \mathbf{p}_1)$ and $(R_2, \mathbf{p}_2)$

Output: Distance feature vector used by nearest-neighbor query

```
1: if  $\text{metric} = \text{feature}$  then
2:    $\Phi_1 \leftarrow \text{stacked world points of local bar feature set under } \mathcal{C}_1.\text{pose}$ 
3:    $\Phi_2 \leftarrow \text{stacked world points of local bar feature set under } \mathcal{C}_2.\text{pose}$ 
4:   return  $\|\Phi_2 - \Phi_1\|_2$ 
5: else
6:    $\Delta\mathbf{p} \leftarrow \mathbf{p}_2 - \mathbf{p}_1$ 
7:    $\alpha_x \leftarrow \angle(R_1[:, 0], R_2[:, 0])$ 
8:    $\alpha_y \leftarrow \angle(R_1[:, 1], R_2[:, 1])$ 
9:    $\alpha_z \leftarrow \angle(R_1[:, 2], R_2[:, 2])$ 
10:  return  $[\Delta p_x, \Delta p_y, \Delta p_z, \alpha_x, \alpha_y, \alpha_z]$ 
11: end if
```

Algorithm 11a – Feature-Point Construction for Feature Distance

Purpose: build stable local bar feature points from AABB corners for feature-metric comparisons.

Algorithm 18 COMPUTEBARFEATUREPOINTS(**target_bar**)

Input: Bar body handle in PyBullet

Output: Local-frame feature-point set $\mathcal{P}_{local}$ or $\emptyset$

```
1:  $T_{bar}^W \leftarrow \text{GETPOSE}(\text{target\_bar})$ 
2:  $[a_{min}, a_{max}] \leftarrow \text{GETAABB}(\text{target\_bar})$ 
3: if  $a_{min} = \emptyset$  or  $a_{max} = \emptyset$  then
4:   return  $\emptyset$ 
5: end if
6: Construct 4 world AABB corner points:
7:  $\mathbf{c}_1 = [a_x^-, a_y^-, a_z^-]$ ,  $\mathbf{c}_2 = [a_x^-, a_y^+, a_z^+]$ ,  $\mathbf{c}_3 = [a_x^+, a_y^-, a_z^+]$ ,  $\mathbf{c}_4 = [a_x^+, a_y^+, a_z^-]$ 
8:  $T_W^{bar} \leftarrow (T_{bar}^W)^{-1}$ 
9:  $\mathcal{P}_{local} \leftarrow []$ 
10: for each  $\mathbf{c}_i \in \{\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3, \mathbf{c}_4\}$  do
11:    $\mathbf{p}_i^{local} \leftarrow T_W^{bar} \mathbf{c}_i$ 
12:    $\mathcal{P}_{local}.\text{append}(\mathbf{p}_i^{local})$ 
13: end for
14: return  $\mathcal{P}_{local}$ 
```

Design Rationale

Design Choice	Rationale
Plan in task space (bar pose SE(3))	Avoids the combinatorial explosion of 12-DOF joint sampling while naturally encoding the object-centric constraint.
Multiple IK seeds per Capsule rung	Captures all kinematic branches at each Cartesian waypoint, enabling the ladder graph to find a globally smooth joint trajectory.
Lazy rung expansion	Full multi-seed IK is deferred until a topologically promising path is found by RRT-Connect, avoiding wasted computation on discarded branches.
Ladder graph + DFS search	Decouples geometric path topology (found by RRT) from optimal joint-config selection (found by graph search), making each sub-problem tractable independently.
Rigid constraint via projection	The left arm is never sampled independently; it is always solved from the right arm pose, guaranteeing the grasp constraint is satisfied at every configuration in the tree.
C-space shortcut smoothing	Post-RRT shortcutting in joint space (interpolated via right-arm linear interpolation + left-arm re-projection) reduces path length without breaking the dual-arm constraint.

Original Design Intent

This section records the original design sketch from which the implementation was derived. It serves as the reference baseline for the gap analysis that follows.

Core Planner Components

Sample. Draw a random 6-DOF bar pose with respect to the robot base frame. *Do not* solve IK at sampling time — the sampled Capsule carries only a pose, no joint configuration. IK is deferred to the extend step, where a good seed is available from the preceding tree node.

Extend. Cartesian-interpolate between two bar poses. The first waypoint is assumed to already carry a valid IK solution (inherited from the nearest tree node); use that configuration to warm-start the IK solver for the next waypoint (iterative, seed-chained IK). If any waypoint fails IK, stop extending and return only the partial path of waypoints for which a valid joint configuration was found.

Collision. Because each Capsule carries exactly one IK solution, collision checking reduces to: set the robot to that joint configuration and test for self-collision and environment collision. This check is entirely local to the Capsule — it has *no* dependence on the Capsule’s parent node in the tree.

Distance. Measure the pose difference between two bar Cartesian poses. A direct weighted sum of position and orientation can be awkward to tune because the two quantities have different units (metres vs. radians). An easier alternative, inspired by practice in reinforcement learning, is to pick a small set of *feature points* on the object (e.g. the corners of a bounding box), compute their world-frame positions for each pose, and use the sum of squared Euclidean distances. This yields a purely metric (metres-only) distance with no unit mismatch.

Ladder Graph (Post-RRT Joint-Space Selection)

Once the task-space RRT-Connect finds an initial path (a sequence of bar poses), the joint-continuity problem is addressed in a second phase:

1. **Expand** each Capsule rung on the path to its full set of IK solutions (multiple IK seeds / delta-angle sweep of the bar pose).
2. **Build** a ladder graph: nodes are IK solutions at each rung; edges connect solutions on adjacent rungs that are within a joint-distance threshold.
3. **Search** the ladder graph (e.g. DFS or shortest-path) for the minimum-cost joint-continuous sequence.

Because the ladder graph is built only for the single extracted path, the number of IK evaluations is much smaller than if multi-IK were computed for every RRT node.

Staged Development Order

1. **Stage 1 — Floating-bar RRT.** Disable IK computation (Capsules carry no joint config) and disable collision checking (collision function always returns **False**, or checks only static

bar-vs-obstacle collisions at a fixed robot configuration). Expected result: a correct RRT planner operating purely in task space. Verify by plotting the RRT tree in the workspace and confirming it fills the reachable region.

2. **Stage 2 — IK on, collision off.** Enable IK computation in the extend function; the planner now builds a path in which every waypoint has a valid dual-arm joint configuration. Collision checking remains disabled. Verify by inspecting the tree's joint configurations and checking that the dual-arm constraint is satisfied at each node.
3. **Stage 3 — IK on, collision on.** Enable full self-collision and environment-collision checking. Compare planning success rates and tree shapes across stages.

Observability and Debugging Goals

- **Workspace tree visualisation.** Plot the RRT tree's Cartesian bar poses in the pybullet scene at each stage so the spatial exploration pattern is visible.
- **Failure distribution analysis.** Run random restarts with a fixed per-attempt time budget and distinct random seeds. Record, for each attempt, whether the failure occurred at the task-space level (RRT did not connect), IK level (no valid config found), or collision level (all paths are blocked). Visualise the failure distribution across attempts to identify the dominant bottleneck.
- **Per-stage comparison.** The three development stages are specifically designed so that differences in tree structure and planning success rate between stages directly reveal the contribution of each constraint (IK reachability, collision avoidance) to overall planning difficulty.

Gap Analysis: Design Intent vs. Zihao’s Implementation

This section compares the original design sketch against the current implementation, identifies deviations, and discusses implications for performance and debuggability.

Deviation 1 — Sampling: Matches (minor note)

Intent: Do not solve IK at sample time; samples are pure pose draws.

Implementation: `plan()` passes `enable_ik=False` to `_get_sample_fn`, so sampled Capsules always have `config=[]`. **Status:** ✓ **correct.**

Minor gap: The `enable_ik` default is `True`, making it easy to accidentally re-enable IK during sampling. There is no single top-level flag that cleanly selects among the three intended development stages.

Deviation 2 — Extend / Partial Path: Matches but Fragile

Intent: On IK failure mid-extension, return the partial Cartesian path that has valid IK solutions and stop.

Implementation: `_get_extend_fn` yields an empty Capsule (`config=[]`) at the failure point, then breaks. `extend_towards_capsule` uses `takewhile(not collision_fn, extend)`, and since `collision_fn` returns `True` for any empty-config Capsule, the `takewhile` automatically stops before the failure. **Status:** ✓ **correct outcome.**

Risk: The mechanism is *implicit*: the collision function doubles as a “no-IK sentinel.” Any future change to collision semantics could break partial-path logic silently, with no type or assertion to catch it.

Deviation 3 — Collision Check: Matches ✓

Intent: Collision check uses only the single IK config stored in the Capsule; no dependence on the parent node.

Implementation: `collision_fn(c.config[0])` — full self-collision plus environment check, parent-independent. **Status:** ✓ **correct.**

Note: The floating-body collision mode (check bar pose directly, set robot to a fixed config) is present but commented out. Stage 1 of development was attempted but not exposed as a clean toggle.

Deviation 4 — Distance Metric: Partial Deviation

Intent: Pose difference between bar Cartesian poses. The bounding-box / feature-point alternative (sum of 3-D distances of object corners) was proposed as an easier-to-tune option.

Implementation: A 6-D vector $[\Delta p_x, \Delta p_y, \Delta p_z, \alpha_x, \alpha_y, \alpha_z]$ whose ℓ_2 norm is used for nearest-neighbour queries. **Status:** × **feature-point variant not implemented.**

Performance impact: Position (metres) and rotation (radians) are mixed with equal weight, so the nearest-neighbour heuristic is biased in a workspace-size-dependent way. When the reachable workspace is small (<1 m), rotation terms dominate; when it is large, translation dominates. Bounding-box feature points would give a purely metric (metres-only) distance that is easier to reason about and tune.

Deviation 5 — Staged Development / Debug Flags: Missing

Intent: Three cleanly switchable stages accessible via a single top-level flag:

1. Floating-bar RRT (no IK, no collision).
2. IK on, collision off.
3. IK on, collision on.

Implementation: `enable_ik` exists per-function, but there is no global `enable_collision` flag, no documented stage enumeration, and the floating-body collision path is commented out. **Status:** × **missing**.

Impact on debugging: When planning fails it is impossible to isolate whether the failure is at the *task-space* level (RRT cannot connect two bar poses), the *IK* level (poses are reachable but no valid joint config exists), or the *collision* level (a valid path exists in free space but is collision-blocked). Without staged flags, each of these hypotheses requires code changes rather than a flag flip.

Deviation 6 — Failure Distribution Logging: Missing

Intent: For each random restart, log where failure occurred and visualise the distribution across attempts.

Implementation: The `max_attempts` outer loop silently continues on failure with no attribution (RRT vs. ladder-graph) and no seed tracking. **Status:** × **missing**.

Deviation 7 — Ladder Expansion Uses `delta=0.0`: Bug

Intent: After finding a path, expand each rung to *all* possible IK solutions to give the ladder-graph search the widest choice.

Implementation: `expand()` calls `create_valid_confs(delta=0.0, max_attempts=20)`. With `delta=0`, `np.linspace(0, 0, 20)` produces 20 identical zero-rotation perturbations, so the IK solver is called 20 times on the *same* bar pose. **Status:** × **effective bug (unintentional no-op delta sweep)**.

Performance impact: Rungs receive very few distinct IK configurations (the IK solver may not converge to different branches from the same seed/pose), so inter-rung edges are sparse and the DFS frequently finds zero feasible paths even when joint-continuous paths exist. This causes the planner to discard good task-space paths and restart RRT unnecessarily. Changing to `delta= $\pi/4$` or similar would explore the IK manifold properly.

Deviation 8 — Capsule Struct is Dual-Purpose

Intent: During planning, each Capsule holds one IK solution (tree node); multi-solution expansion happens only on the extracted path (ladder rung).

Implementation: The same `Capsule` class serves both roles. The `parent` pointer means *tree connectivity* during RRT and *rung adjacency* during ladder search. The only distinction is the “fast-skip” check `len(config) > 1` inside `rrt_connect_capsule`. **Status:** ~ **works but confusing**.

Impact: Accidental mutation of a tree-node Capsule’s config during the expand phase can corrupt the tree. Separate `TreeNode` and `LadderRung` types would make each role explicit and eliminate the fast-skip hack.

Deviation 9 — Ladder Search Inside the RRT Loop

Intent: Two clearly separated phases: (1) RRT finds a path, (2) ladder graph is expanded and searched.

Implementation: `expand_path` and `configs_capsule` run *inside* the RRT iteration loop on every connection event, before any path is confirmed to be the best one. If the DFS fails, the loop restarts RRT. **Status:** ~ **functional but expensive and hard to profile.**

Impact: Each RRT connection event pays the full multi-IK expansion cost. Profiling cannot easily separate “time in RRT” from “time in ladder search.” The two-phase separation in the original design was meant to keep these costs visible and individually tunable.

Summary

Aspect	Intent	Status	Key risk
No IK at sample time	✓	✓	Easy to regress (default=True)
Partial extend path	✓	✓	Implicit via collision sentinel
Parent-independent collision	✓	✓	—
Pose-based distance (un-weighted)	✓	×	Unit mismatch, biased NN
Feature-point distance option	proposed	×	Not implemented
3 staged dev. flags	✓	×	Cannot isolate failure mode
Failure distribution logging	✓	×	No restart statistics
Tree workspace visualisation	✓	~	<code>draw_fn</code> exists but not systematic
$\delta > 0$ in ladder expand	implied	×	Sparse ladder, frequent DFS failure
Separate tree/ladder types	implied	~	Single struct, dual role

Proposed Investigation Pathways

Path A — Fix debug scaffolding first. Add `enable_ik: bool` and `enable_collision: bool` to `plan()`. Restore `floating_collision_fn` as stage-1 collision. Verify the task-space RRT explores the workspace correctly before adding IK and collision constraints.

Path B — Fix the distance metric. Implement a feature-point distance: choose 4 corners of the bar’s bounding box, compute their world-frame positions for each Capsule pose, and use the sum of squared 3-D distances. This is purely in metres, eliminates unit mixing, and directly matches the RL intuition cited in the design notes. Compare nearest-neighbour quality and tree-growth shape before and after.

Path C — Fix ladder expansion (δ). Change `expand()` to use `delta=np.pi/4` (or expose as a parameter). Measure: (a) mean number of distinct IK solutions per rung, (b) DFS success rate (ladder failures vs. RRT failures). This is the highest-leverage single-line fix.

Path D — Separate tree nodes from ladder rungs. Introduce a `LadderRung` dataclass with an explicit multi-config list and connection map. Produce it only inside `expand_path`, not inside `RRT`. This eliminates the fast-skip hack and the dual-role Capsule ambiguity.

Path E — Add failure attribution and visualisation. For each attempt in the restart loop, record: did RRT connect? did expand succeed? did DFS find a path? Print a summary after all attempts and, when `use_draw=True`, draw the final extent of both RRT trees in the workspace so failure geometry is visible.